

CERTIFICATION CONCEPTEUR DÉVELOPPEUR D'APPLICATIONS

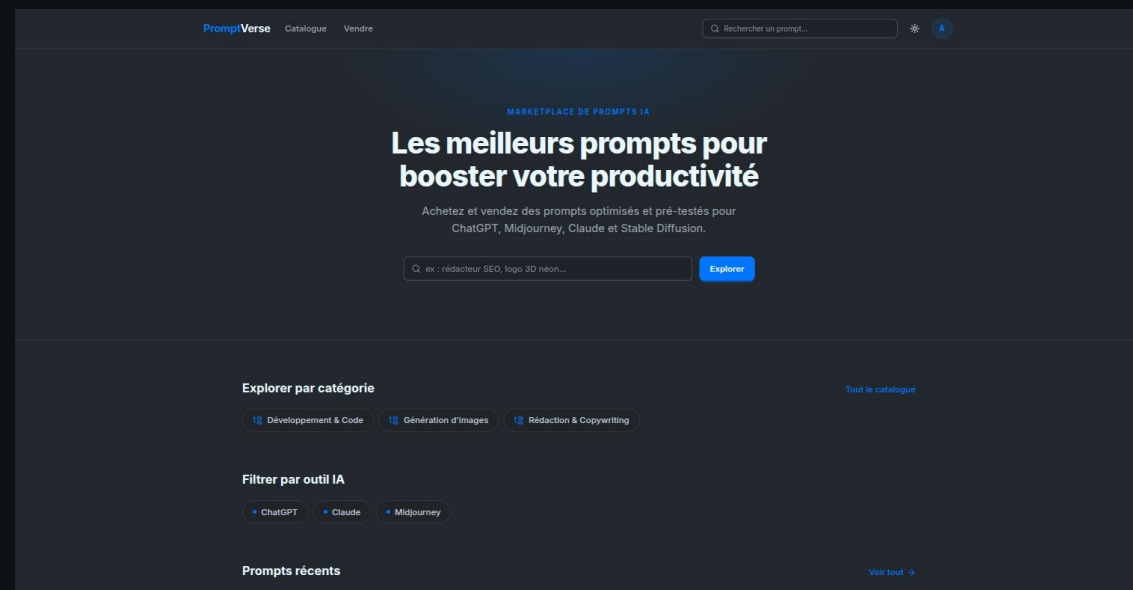
PromptVerse

La marketplace de prompts pour les IA génératives

promptverse.biz

Abduljaber AL OMRAN

Simplon · Formation CDA · 2026



Sommaire

01 Le projet et l'équipe

02 Organisation et outils

03 Conception

04 Architecture et back-end

05 Front-end

06 Sécurité

07 Tests

08 DevOps et déploiement

09 Veille, évolutions et démo

Introduction



Abduljaber AL OMRAN

Apprenant Concepteur Développeur d'Applications

Simplon · 2025 – 2026



Titre visé

Concepteur Développeur d'Applications — niveau 6



Projet chef-d'œuvre

PromptVerse, réalisé en équipe de trois



En production

<https://promptverse.biz>

PromptVerse, c'est quoi ?

Une marketplace où chacun peut vendre ses prompts optimisés pour les IA génératives, et acheter ceux des autres pour gagner du temps.



Catalogue

Prompts filtrables par catégorie et par outil IA (ChatGPT, Midjourney, Claude...)



Publication

Tout utilisateur publie ses prompts avec prix, aperçu et images



Achat

Paieement Stripe Checkout ; les prompts à 0 € sont gratuits



Administration

Gestion des utilisateurs, des rôles, des catégories et des outils IA

Règle clé : le contenu payant d'un prompt reste invisible tant que l'achat n'est pas confirmé.

L'équipe

AA

Abduljaber

Conception · Développement

AM

Amine

Conception · Développement

AN

Anis

Conception · Développement



Formateur Simplon : briefs de sprint, suivi et validation des livrables

Communication et organisation



Trello

Backlog par sprint, user stories



GitHub

Branches dev / main / deploy



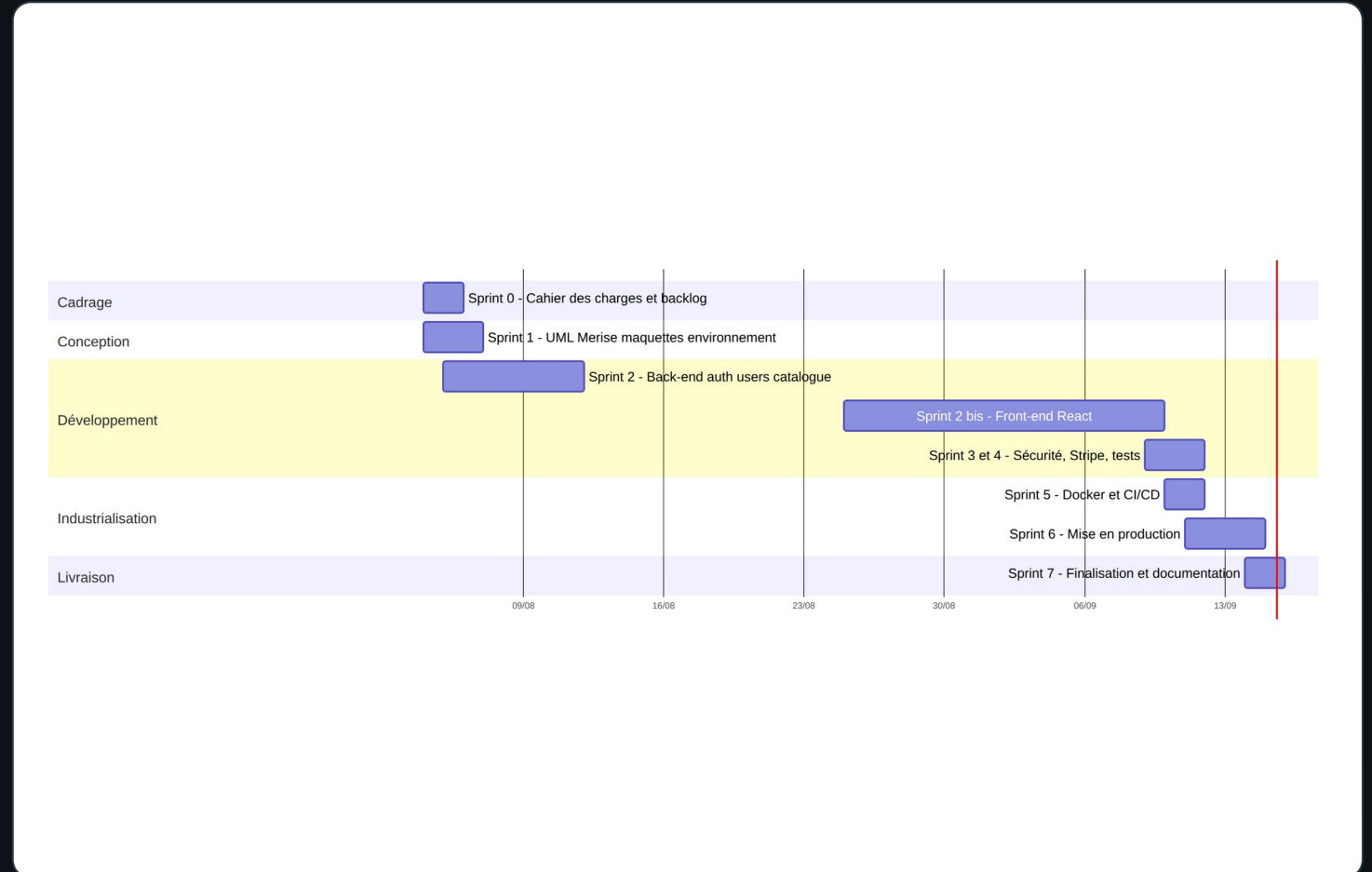
OpenSpec

Spécification avant le code



Scrum

8 sprints, revue à chaque fin



Planning des sprints (reconstitué depuis l'historique Git)

Stack et outils

Front-end



React 19



TypeScript



Vite



TanStack Query



Tailwind + daisyUI



React Router

Back-end



NestJS 11



TypeORM



MySQL 8.4



JWT + bcrypt



Stripe

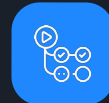


Jest

DevOps et outils



Docker



GitHub Actions



Nginx



Let's Encrypt



Swagger



Git

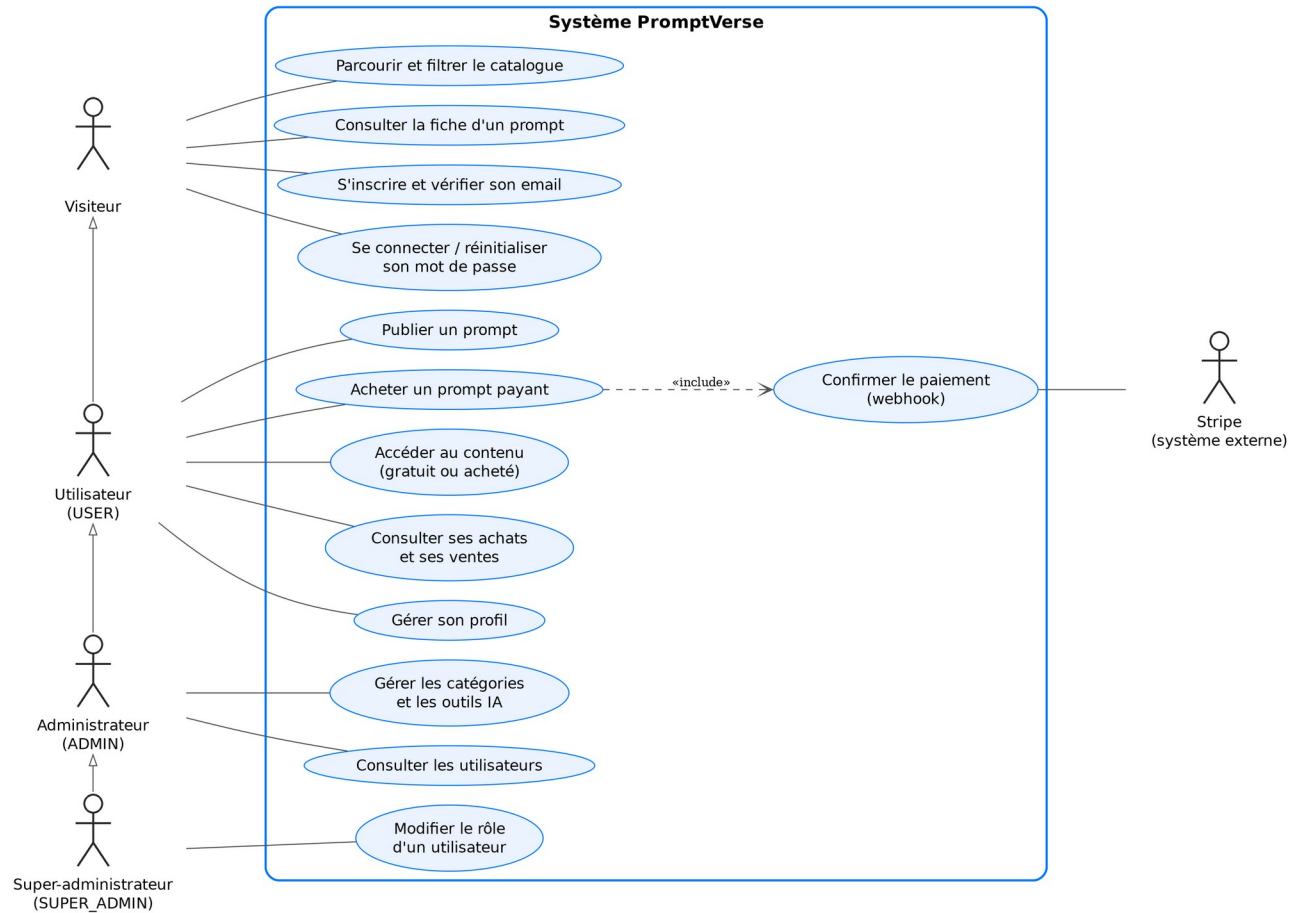
03

Conception

Des besoins aux modèles, avant la première ligne de code



Diagramme de cas d'utilisation

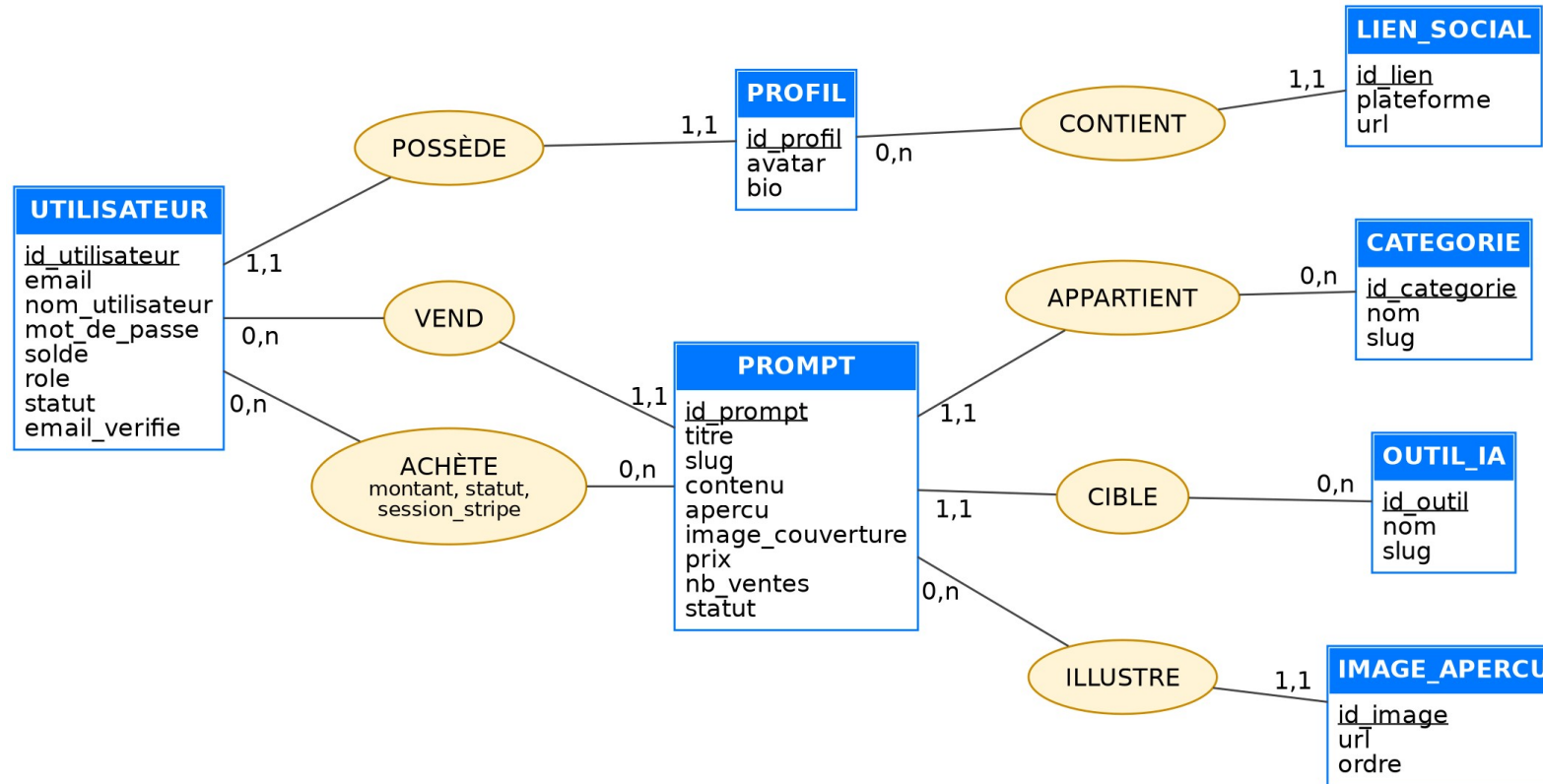


4 acteurs + Stripe (système externe)

Héritage visiteur → USER → ADMIN
→ SUPER_ADMIN

«include» l'achat inclut la confirmation par webhook

Modèle conceptuel de données



ACHÈTE porte ses propres données (montant, statut, session Stripe) → devient la table purchases dans le MLD.

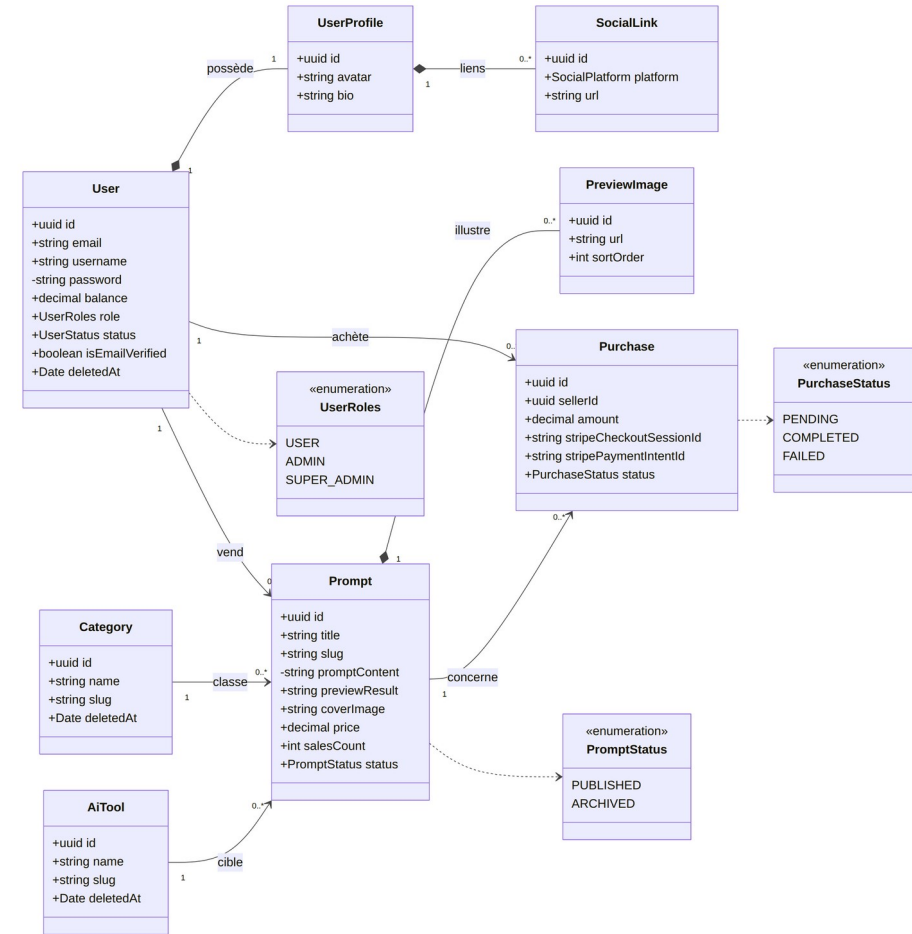
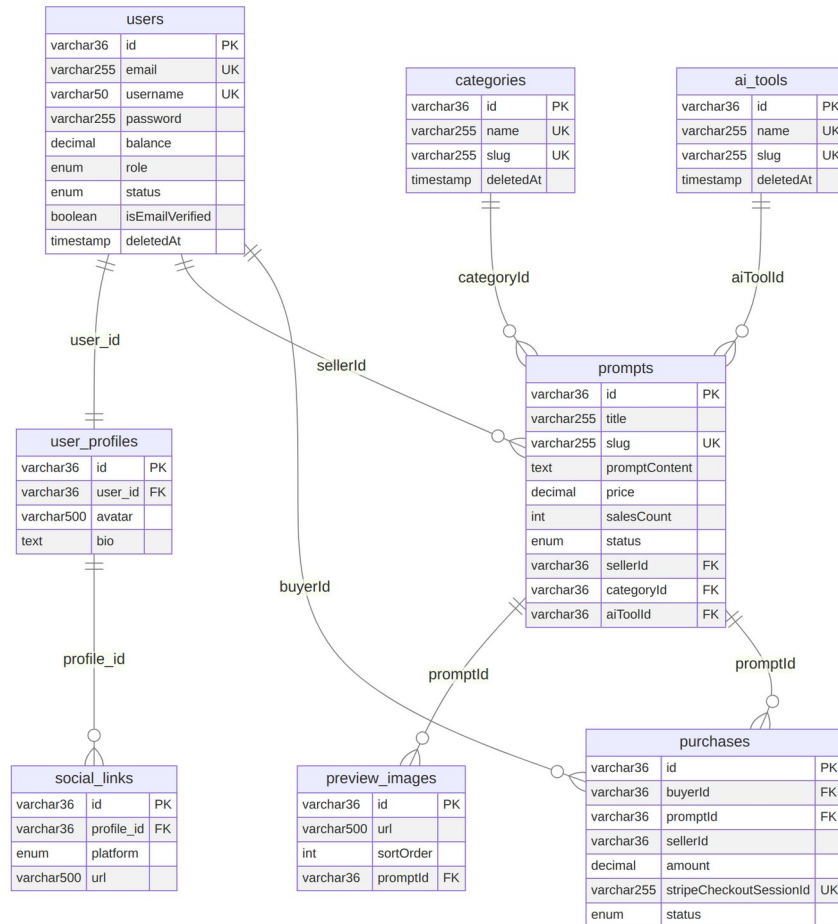
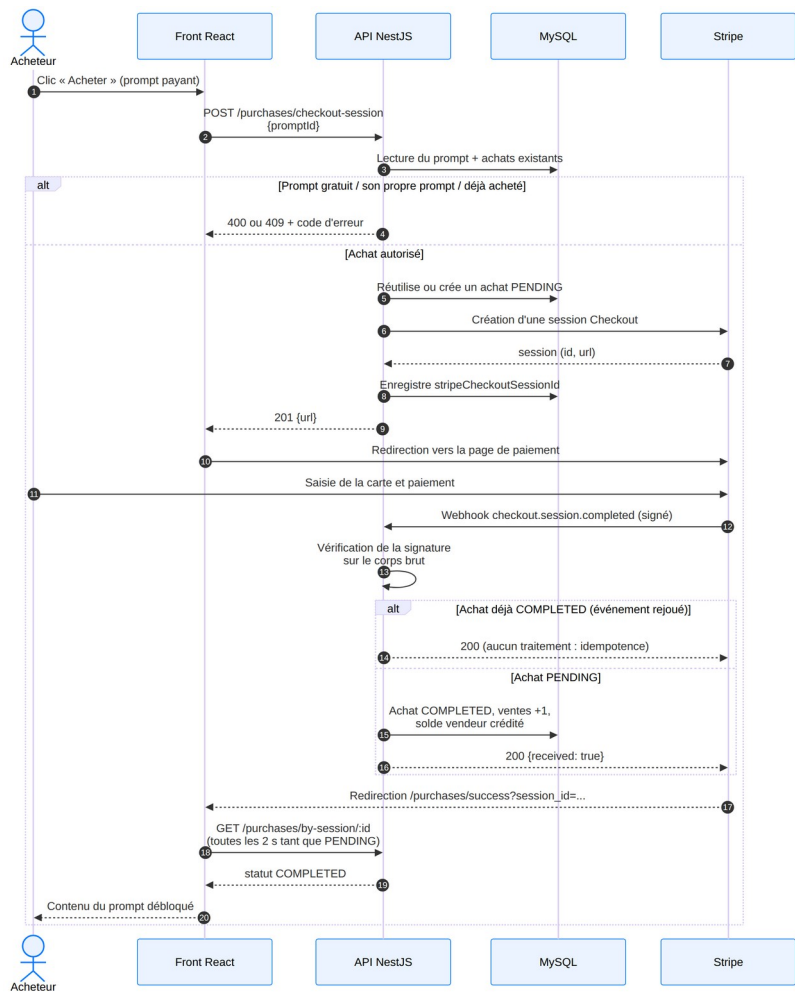
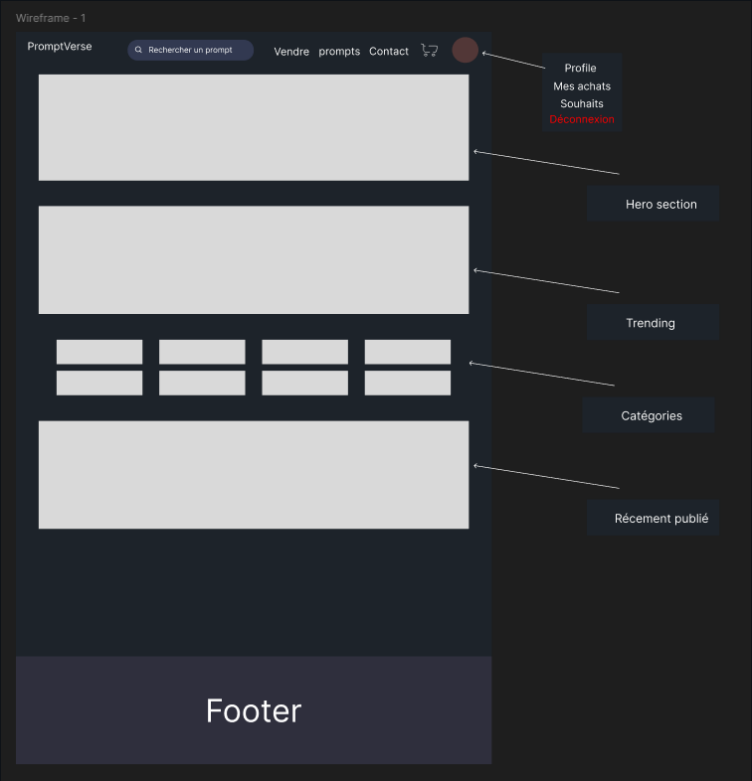


Diagramme de séquence : l'achat

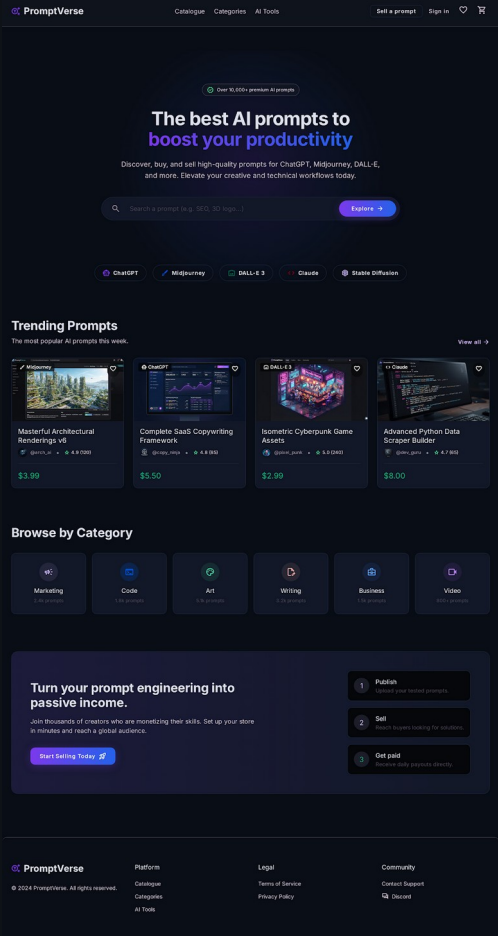


- 1 Contrôle des règles métier
- 2 Achat PENDING + session Stripe
- 3 Paiement sur la page Stripe
- 4 Webhook signé → COMPLETED
- 5 Contenu débloqué

Du wireframe à l'application



Wireframe



Maquette haute fidélité



Rendu final

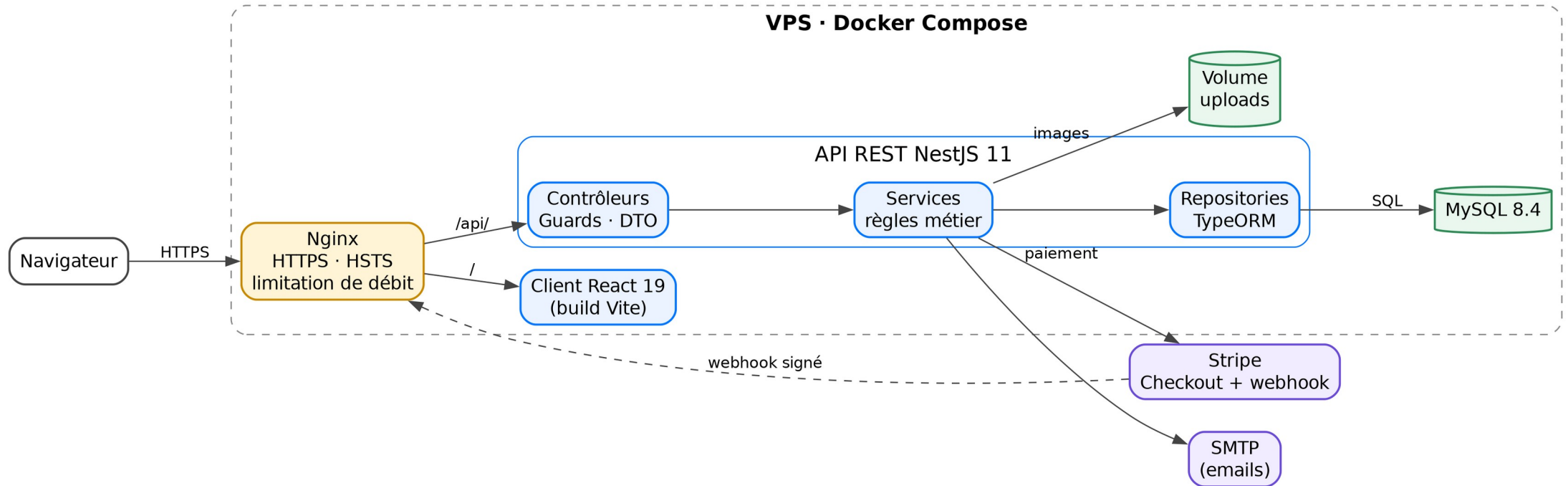
04

Architecture et back-end

API REST NestJS organisée en couches



Architecture de l'application



Découplée : le front ne parle qu'à l'API

Un seul point d'entrée : Nginx en HTTPS

Sécurité centralisée : toutes les règles côté serveur

Circulation d'une requête dans l'API



Exemple : l'achat d'un prompt

```
POST /api/v1/purchases/checkout-session { "promptId": "..." }  
→ AuthGuard (cookie JWT) → ValidationPipe (CreateCheckoutSessionDto : UUID)  
→ PurchasesController.createCheckoutSession()  
→ PurchasesService (règles métier) → Repository<Purchase> → MySQL
```


Authentification : JWT en cookie httpOnly

auth.controller.ts (extrait simplifié)

```
@Post('login')
@Throttle({
  short: { limit: 2, ttl: 1000 },
  medium: { limit: 5, ttl: 60000, blockDuration: 900000 },
})
@Public()
async login(@Body() loginDto: LoginDto,
  @Res({ passthrough: true }) res: Response) {
  const { accessToken } = await this.authService.login(loginDto);

  res.cookie('access_token', accessToken, {
    httpOnly: true,           // illisible en JS
    secure: isProduction,     // HTTPS uniquement
    sameSite: 'lax',
    maxAge: 1000 * 60 * 60 * 24, // 24 h
  });
  return { message: 'Login successful' };
}
```

bcrypt coût 10, mot de passe jamais renvoyé

Email vérifié obligatoire pour se connecter

Cookie httpOnly protège le jeton en cas de XSS

Guard global toute route est privée sauf @Public()

Throttling contre la force brute

Règle métier : protéger le contenu payant

prompts.service.ts — findOneBySlug()

```
const isOwner = currentUserId === prompt.sellerId;
const isFree = PromptsService.isFree(prompt.price);
const hasPurchased = !isOwner && !isFree && currentUserId
  ? await this.hasCompletedPurchase(currentUserId, prompt.id)
  : false;
const canViewFullContent = isOwner || isFree || hasPurchased;

return {
  ...prompt,
  promptContent: canViewFullContent
    ? prompt.promptContent
    : null,
};
```

Qui voit le contenu ?

Visiteur	null
Utilisateur sans achat	null
Vendeur	visible
Achat COMPLETED	visible
Prompt gratuit (0 €)	visible

Et dans les listes du catalogue, la colonne n'est même pas lue en base (sélection explicite des colonnes).

Paieement Stripe et webhook idempotent

purchases.service.ts — createCheckoutSession()

```
if (PromptsService.isFree(prompt.price))  
  throw new BadRequestException({ code: ErrorCodes.PROMPT_IS_FREE });  
if (prompt.sellerId === buyerId)  
  throw new BadRequestException({ code: ErrorCodes.CANNOT_PURCHASE_OWN_PROMPT });  
if (await this.promptsService.hasCompletedPurchase(buyerId, promptId))  
  throw new ConflictException({ code: ErrorCodes.PROMPT_ALREADY_PURCHASED });
```

purchases.service.ts — completePurchase()

```
// Stripe peut renvoyer le même événement plusieurs fois  
if (purchase.status === PurchaseStatus.COMPLETED) return;  
  
purchase.status = PurchaseStatus.COMPLETED;  
await this.purchasesRepository.save(purchase);  
await this.promptsService.incrementSalesCount(purchase.promptId);  
seller.balance = Number(seller.balance) + Number(purchase.amount);
```



Signature vérifiée

Le webhook est public mais vérifié sur le corps brut de la requête (rawBody). Une fausse confirmation est rejetée avec une erreur 400.

05

Front-end

Application monopage React 19 + TypeScript



Organisation du front-end



pages/

Un composant par écran



components/

Composants réutilisables par domaine



common/api/

Axios, cookie envoyé automatiquement



hooks/

TanStack Query : données serveur



common/store/

Zustand : session, toasts, UI

Pourquoi TanStack Query + Zustand ?

- Cache, chargement et erreurs gérés automatiquement
- Séparation claire : données serveur $\neq$ état de l'interface
- Beaucoup moins de code répétitif
- Plus léger et plus simple que Redux pour ce besoin

Routes protégées et formulaires validés

ProtectedRoute.tsx

```
export function ProtectedRoute() {
  const { status } = useAuth();
  const location = useLocation();

  if (status === "pending")
    return <PageLoader label="Vérification..." />;
  if (status === "unauthenticated")
    return <Navigate to="/login" replace
      state={{ from: location.pathname }} />;

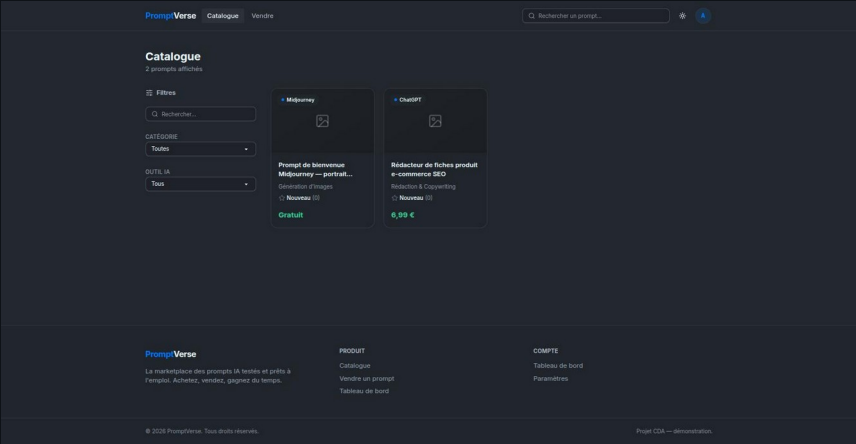
  return <Outlet />;
}
```

RegisterPage.tsx — react-hook-form + zod

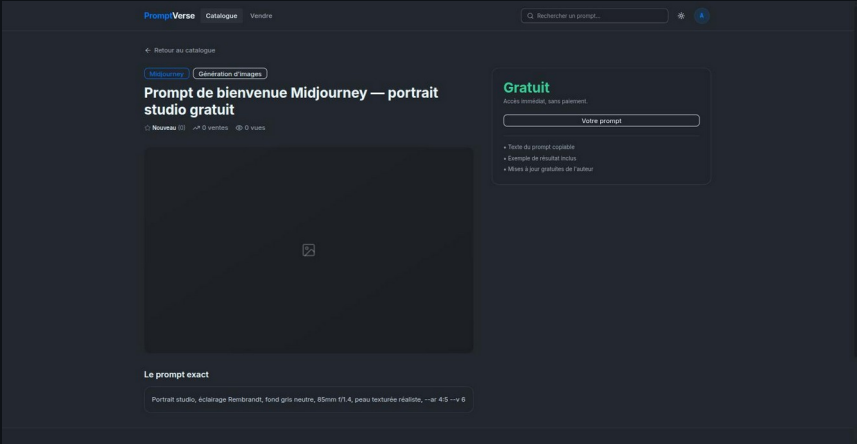
```
const schema = z.object({
  username: z.string().min(3).max(50),
  email: z.string().email("E-mail invalide."),
  password: z.string().min(6),
  confirmPassword: z.string(),
  terms: z.boolean().refine((v) => v === true),
}).refine((d) => d.password === d.confirmPassword, {
  path: ["confirmPassword"],
  message: "Les mots de passe ne correspondent pas.",
});
```

Côté client, ces contrôles servent le confort de l'utilisateur : la vraie sécurité reste dans l'API.

L'application



Catalogue filtrable



Fiche d'un prompt

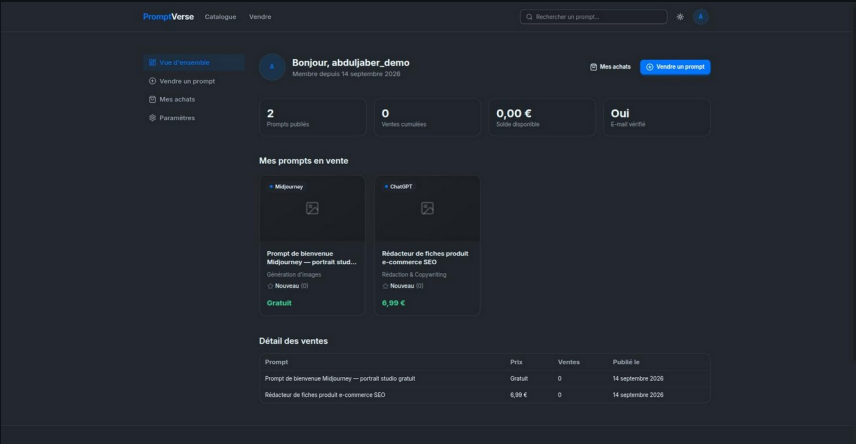
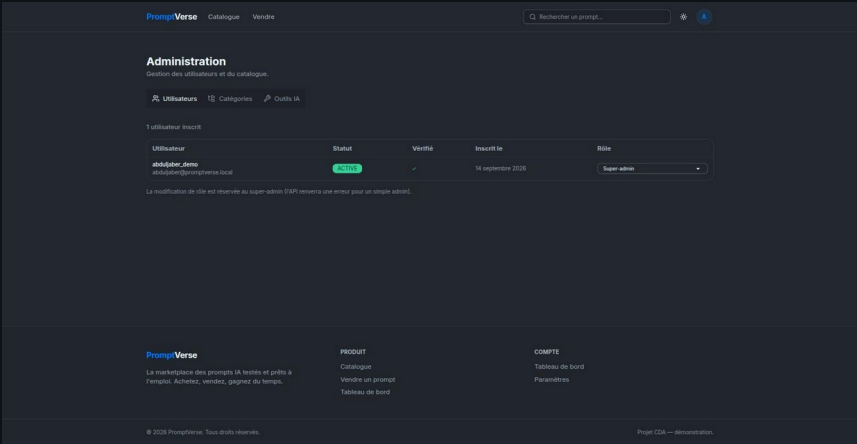


Tableau de bord vendeur



Administration des rôles

06

Sécurité

Pensée dès la conception, pas ajoutée à la fin



Mesures de sécurité



Authentification

JWT signé, cookie httpOnly + secure
+ sameSite, bcrypt



Contrôle d'accès

Guard global + rôles USER / ADMIN /
SUPER_ADMIN



Validation

DTO avec whitelist, champs inconnus
refusés



Limitation de débit

Throttler API + limites Nginx



Injection SQL

Requêtes paramétrées par TypeORM



Transport et en-têtes

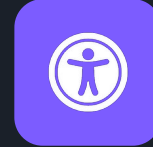
HTTPS, HSTS, Helmet, CORS
restreint

RGPD et accessibilité



Données personnelles

- Collecte minimale : email, pseudo
- Mot de passe haché, jamais en clair
- Aucune donnée bancaire : tout passe par Stripe
- Acceptation des conditions à l'inscription
- Suppression du compte depuis les paramètres



Accessibilité

- Chaque champ a un libellé (label / htmlFor)
- Textes alternatifs sur les images
- Messages d'erreur en texte sous le champ
- Thème sombre à fort contraste
- Audit RGAA complet : à venir

07

Tests

Des critères d'acceptation aux tests automatisés



Tests unitaires et jeu d'essai

18 / 18

tests unitaires Jest réussis

10 / 11

scénarios du jeu d'essai conformes

purchases.service.spec.ts

```
it('is idempotent: a second completed event
  for the same session is a no-op', async () => {
  findOnePurchase.mockResolvedValue({
    ...purchase, status: PurchaseStatus.COMPLETED,
  });

  await service.handleWebhookEvent(Buffer.from('{}'), 'sig');

  expect(savePurchase).not.toHaveBeenCalled();
  expect(usersService.save).not.toHaveBeenCalled();
});
```

Unitaires : services des prompts, des achats et de l'administration, avec des mocks

Jeu d'essai : vraies requêtes sur l'API (achat de son propre prompt, prompt gratuit, webhook falsifié...)

Écart analysé : clé Stripe de test invalide en local

08

DevOps et déploiement

Du git push à la production



Conteneurisation avec Docker



Développement

docker-compose.dev.yml

- api (rechargement à chaud)
- client (Vite)
- database (MySQL 8.4)
- mailpit (emails de test)



Production

docker-compose.prod.yml

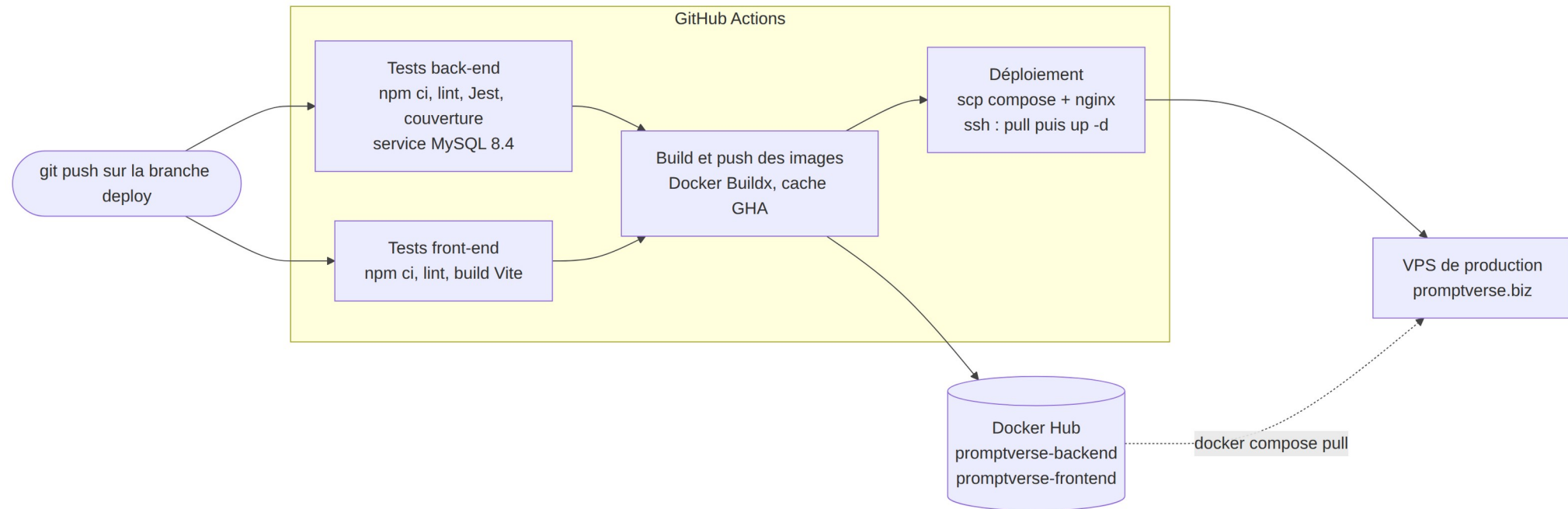
- nginx (HTTPS, reverse proxy)
- api (image Docker Hub)
- client (image Docker Hub)
- database (volume persistant)

Dockerfile multi-stage de l'API

```
# 3 étapes : build → dépendances → image finale
FROM node:22-alpine AS build
RUN npm ci && npm run build
FROM node:22-alpine AS deps
RUN npm ci --omit=dev
FROM node:22-alpine AS production
COPY --from=build /app/dist ./dist
USER node # pas de root

# docker-entrypoint.sh
attendre MySQL → migration:run → node dist/main
```

Pipeline CI/CD GitHub Actions



1 Tests back

lint · Jest · couverture

2 Tests front

lint · build Vite

3 Images

build + push Docker Hub

4 Déploiement

scp + ssh sur le VPS

Mise en production



VPS Linux

Docker Compose de production



Nginx

Seul service exposé (80 / 443)



HTTPS

Certificat Let's Encrypt via Certbot



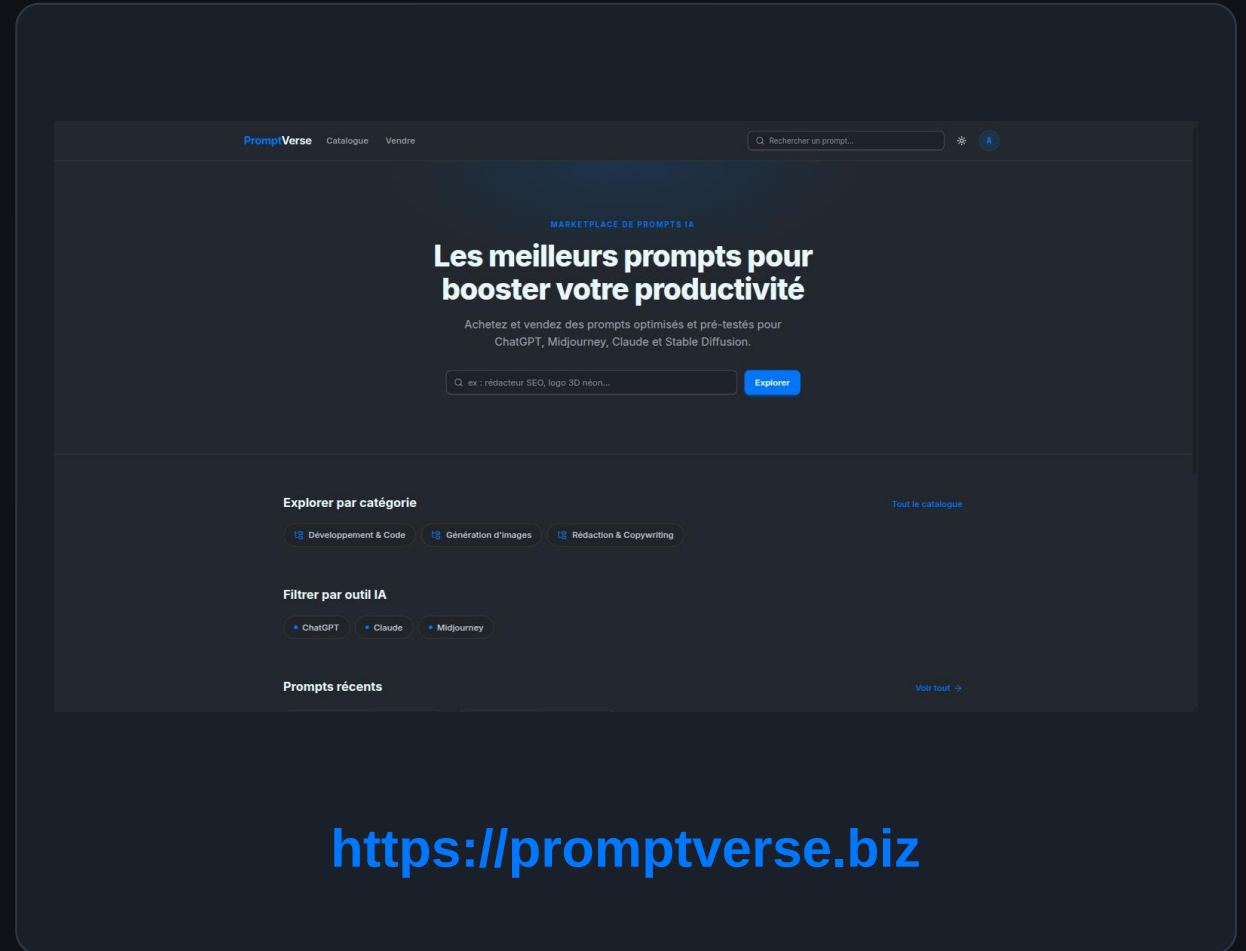
Données

Volumes MySQL et uploads persistants



Retour arrière

Images taguées par commit



<https://promptverse.biz>

Veille sur les vulnérabilités

Sources : OWASP Top 10 · CERT-FR / ANSSI · GitHub Advisory Database · npm audit

0

vulnérabilité côté client

15

alertes côté API (npm audit)



Faible trouvée dans notre code

L'envoi d'images accepte tout type « image/* », y compris le SVG, qui peut contenir du JavaScript (risque de XSS stocké). Le type MIME est en plus déclaré par le client.

Correctif prévu

- Liste blanche : JPEG, PNG, WebP uniquement
- Contrôle de la signature binaire du fichier
- Mise à jour des dépendances + npm audit dans la CI

Évolutions du projet



Avis et notes



Liste de favoris



Recherche plein texte



Reversement aux vendeurs (Stripe Connect)



Tests End-to-End et meilleure couverture



Durcissement : upload, mots de passe, dépendances

DÉMONSTRATION

Place à la démo

- Inscription et vérification de l'email
- Publication d'un prompt
- Achat d'un prompt avec Stripe
- Espace d'administration

promptverse.biz



Conclusion



Tout le cycle

Des besoins à la production, en passant par UML, Merise, le code, les tests et le déploiement



La sécurité dès la conception

Cookie httpOnly, routes privées par défaut, contenu payant masqué, webhook idempotent



Le travail en équipe

Backlog partagé, un dépôt commun, des décisions prises à trois



L'esprit DevOps

Un projet est terminé quand il tourne en production, de façon reproductible

Merci pour votre attention

Place à vos questions



Abduljaber AL OMRAN · PromptVerse · promptverse.biz